

Introducing Workflow Management to Computational Social Science

Andreas Kuepfer
TU Darmstadt

Christian Arnold
University of Birmingham

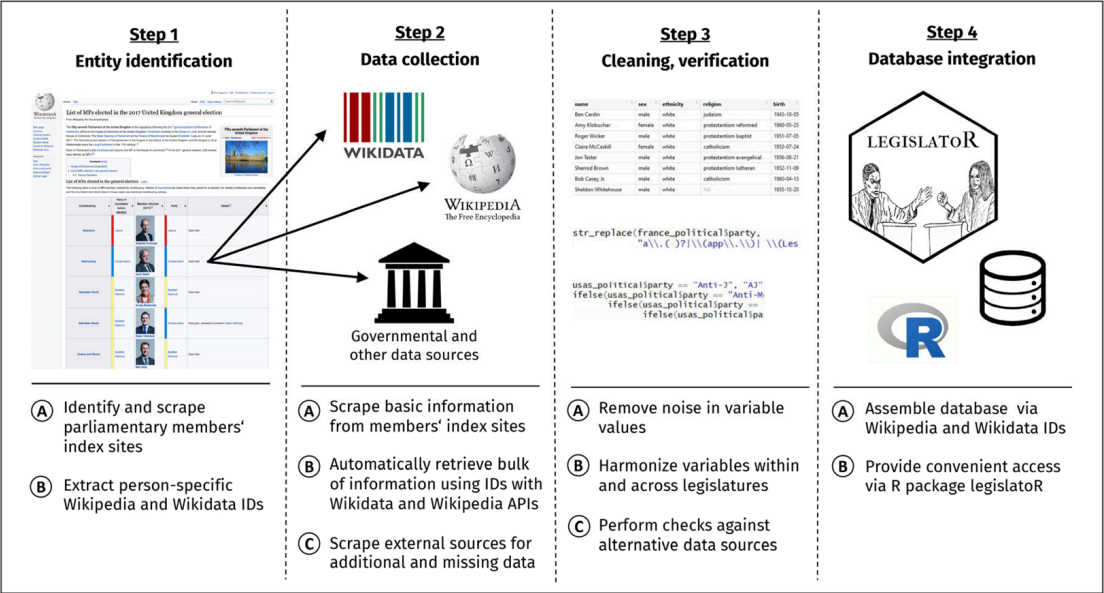
ParliView Workshop (May 12, 2026) | University College Dublin, Ireland

TABLE 1 AI-powered text analysis workflow.

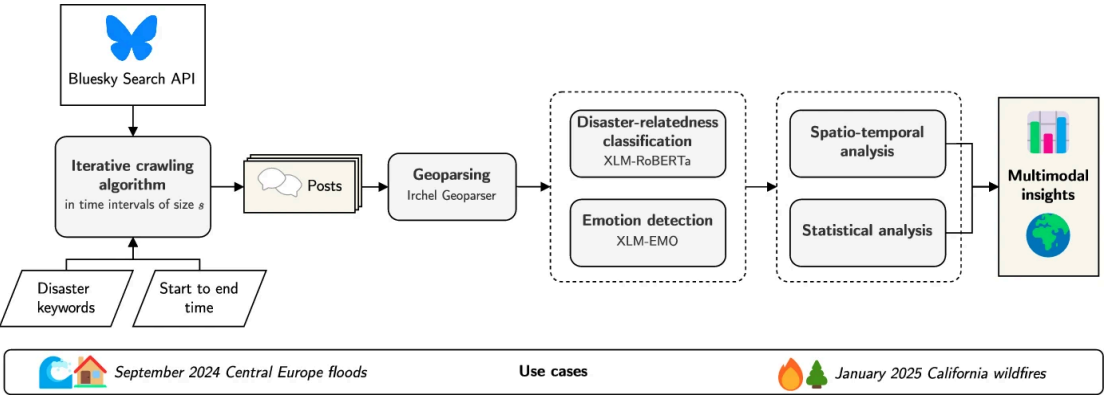
1. Input texts and validation data	
a. Text corpus:	
i. Party manifestos from the Manifesto Project (MP)	
ii. Coalition agreements from Klüver et al. (2023)	
b. Positions on six issue dimensions from expert surveys fielded in election years:	
i. Laver and Hunt (1992); Benoit and Laver (2006)	
ii. Chapel Hill Expert Survey	
c. Issue positions on same six dimensions derived from scaling hand-coded labels:	
i. MP data for party manifestos	
ii. Klüver et al. (2023) data for coalition agreements	
2. Large language model (LLM) authorization	
Establish high-tier API access to three commercial LLMs: OpenAI's GPT-4o, Anthropic's Claude 3.5 Sonnet, Google's Gemini 1.5 Pro	
3. Summarize texts	
a. Present original language texts to LLMs for processing	
b. Prompt LLMs to generate 200–300-word English language summaries of each text's content relating to each of the six issue dimensions.	
4. Scale text summaries	
Prompt LLMs to locate positions of each summary on the specified 7-point scales	
a. for “zero-shot” learning (no examples)	
b. for “few-shot” learning (adding three sample summaries and associated benchmark positions for each scale)	
5. Validation	
a. <i>Concurrent validity</i> : Correlate ensemble means of LLM estimates of party positions with ensemble means of expert judgments from 1b	
b. <i>Reliability</i> : As a benchmark for reliability, compare LLM-expert correlations with expert–expert correlations.	
c. <i>Predictive validity</i> : Compare ensemble means of LLM estimates of coalitions' issue positions with analogous hand-coded estimates from 1c, to see which better accords with a prediction common to almost all models of government formation.	
6. Replication and robustness checks	
Repeat Steps 2–5a to see if results are stable.	
Replicate results using three open-weight LLMs: DeepSeek-V3 (671B), LLaMA-3.3 (70B), and Gemma-3-27B-it.	

Notes: This lists each of the steps in the workflow generating the results reported in this paper.

(Benoit et al., 2026)



(Göbel and Munzert, 2018)



(Hanny et al., 2025)

CSS Pipelines Have Outgrown Their Infrastructure

CSS pipelines have grown:

- Large-n corpora: text, audio, image, video
- Multi-step processing chains involving different models and environments
- Workloads often exceeding a single machine

But infrastructure has not kept pace: mostly still a collection of standalone scripts run by hand.

Six Infrastructure Issues in Current Practice (Adapted from *Mölder et al. (2021)*)

Automation

Scripts executed by hand, in mental order
→ Stale outputs; execution errors go undetected

Scalability

Code designed to fit a laptop
→ Computationally intensive questions stay unanswered

Portability

Hardcoded paths, undeclared dependencies
→ Code that works on one machine breaks silently on another

Readability

Scripts written for one-time use
→ Replication is nominal; reuse often challenging

Technical Documentation

No automatic logging
→ Results cannot be traced to conditions of production

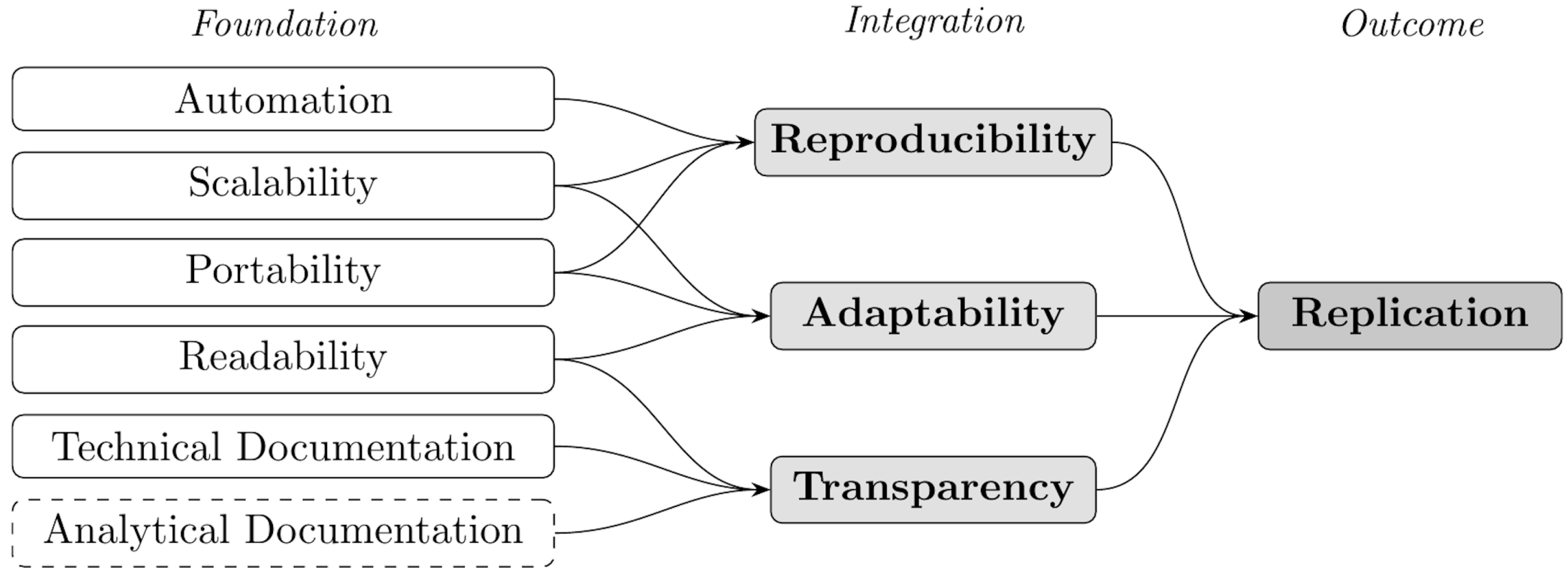
Analytical Documentation

Decisions undocumented in code
→ Researcher degrees of freedom remain invisible

Workflow Management Systems

A systematic solution to an infrastructure problem

Framework: From Six Properties to Full Replication



Workflow Management Systems (WMS) Offer a Systematic Solution

A WMS represents a pipeline as a **dependency graph**: each step declares its **inputs** and **outputs** while the system enforces order, detects stale outputs, and manages environments automatically.

```
1 rule summarize_texts:
2     input:
3         text = "data/{speech}.txt"
4     output:
5         summary = "output/{speech}_summary.txt"
6     script:
7         "scripts/summarize.py"
8
9 rule scale_zeroshot:
10    input:
11        summary = "output/{speech}_summary.txt"
12    output:
13        scores = "output/{speech}_scale_zeroshot.json"
14    script:
15        "scripts/scale_zeroshot.py"
16
17 rule scale_fewshot:
18    input:
19        summary = "output/{speech}_summary.txt",
20        benchmarks = "data/benchmark_examples.json"
21    output:
22        scores = "output/{speech}_scale_fewshot.json"
```

Snakemake as a WMS

- Fastest end-to-end execution among WMS benchmarks (*Larsonneur et al., 2018*)
- 75% of public Snakemake repos have less than 23 rules (*Pohl et al., 2024*)
- Hundreds of citations annually in STEM; social sciences account for just >1% (*Mölder et al., 2021*)

WMS Pays Off Regardless of Where You Run

Working locally

Isolated environments per rule

No more conflicting packages across Python, R, and model inference steps

Shareable by design

A collaborator clones the repo, runs one command, and gets the same result

Audit trail without effort

Every run is logged; reviewers can trace any output to its inputs

Working on a cluster

Zero porting overhead

The same workflow runs on different clusters with a single profile flag

Automatic job submission

WMS submit and monitor many jobs in parallel; you don't manage the queue

Fault tolerance

Failed jobs rerun automatically; completed jobs are never recomputed

Do you need a WMS? A Checklist for Social Scientists

More than one script?

Your pipeline has steps that must run in order

Do you re-run selectively?

You mentally track which outputs are outdated

Does processing take hours?

A crash means starting over or patching by hand

Working with collaborators?

Others need to run and understand your code on a different machine

Considering HPC or cloud?

Your data exceeds what a laptop can handle

Submitting to a journal?

A third party needs to verify your results

Snakemake in Practice

30,000 hours of parliamentary video across 17 German parliaments

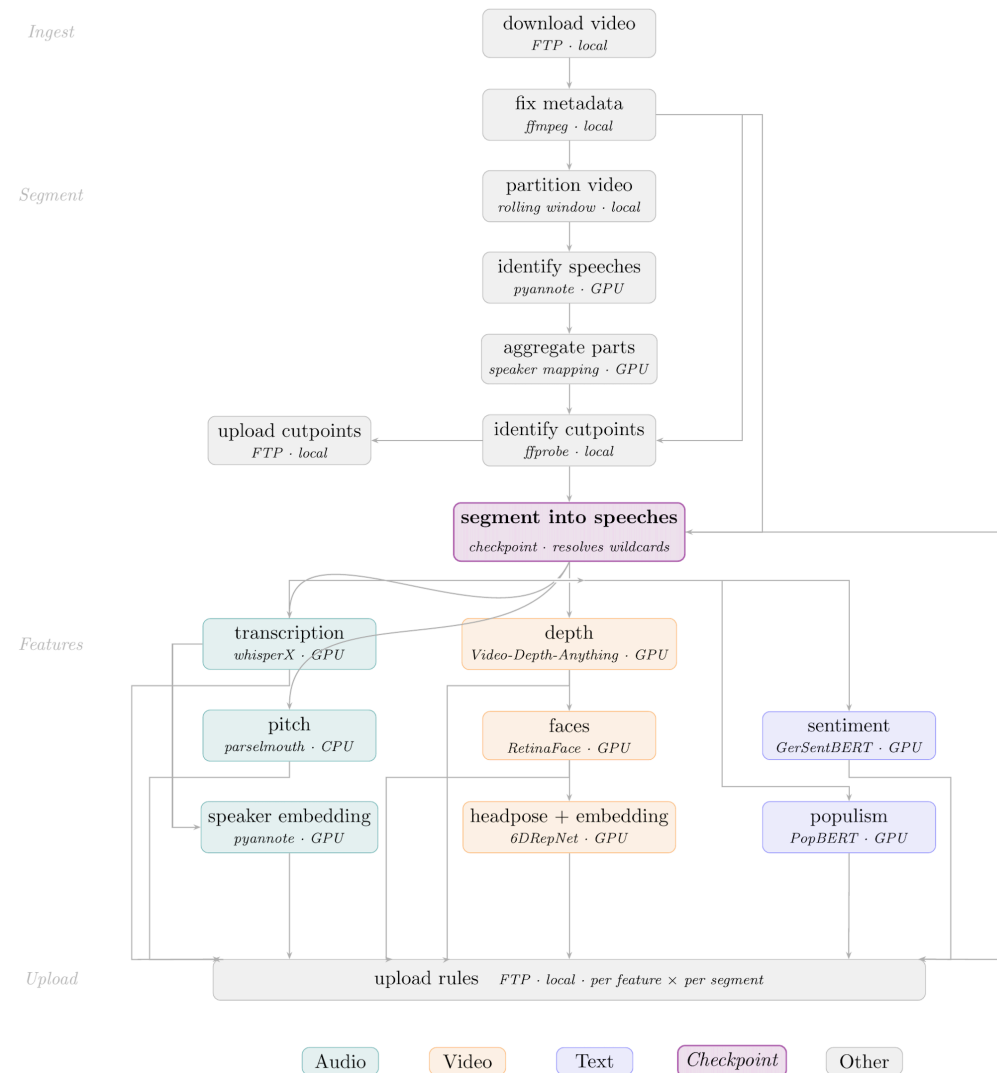
Application: A Snakemake Pipeline for 30,000 Hours of Parliamentary Video

The corpus

- All available speeches from the German Bundestag and all 16 state parliaments
- >30,000 hours (>20TB) of footage
- ~170,000 individual uncut video files
- Heterogeneous across parliaments: format, segmentation, and recording quality vary widely

Performance with Snakemake

~4 min 20 sec end-to-end for 1 hour of raw video



Application: Is Populist Rhetoric Spontaneous or Scripted?

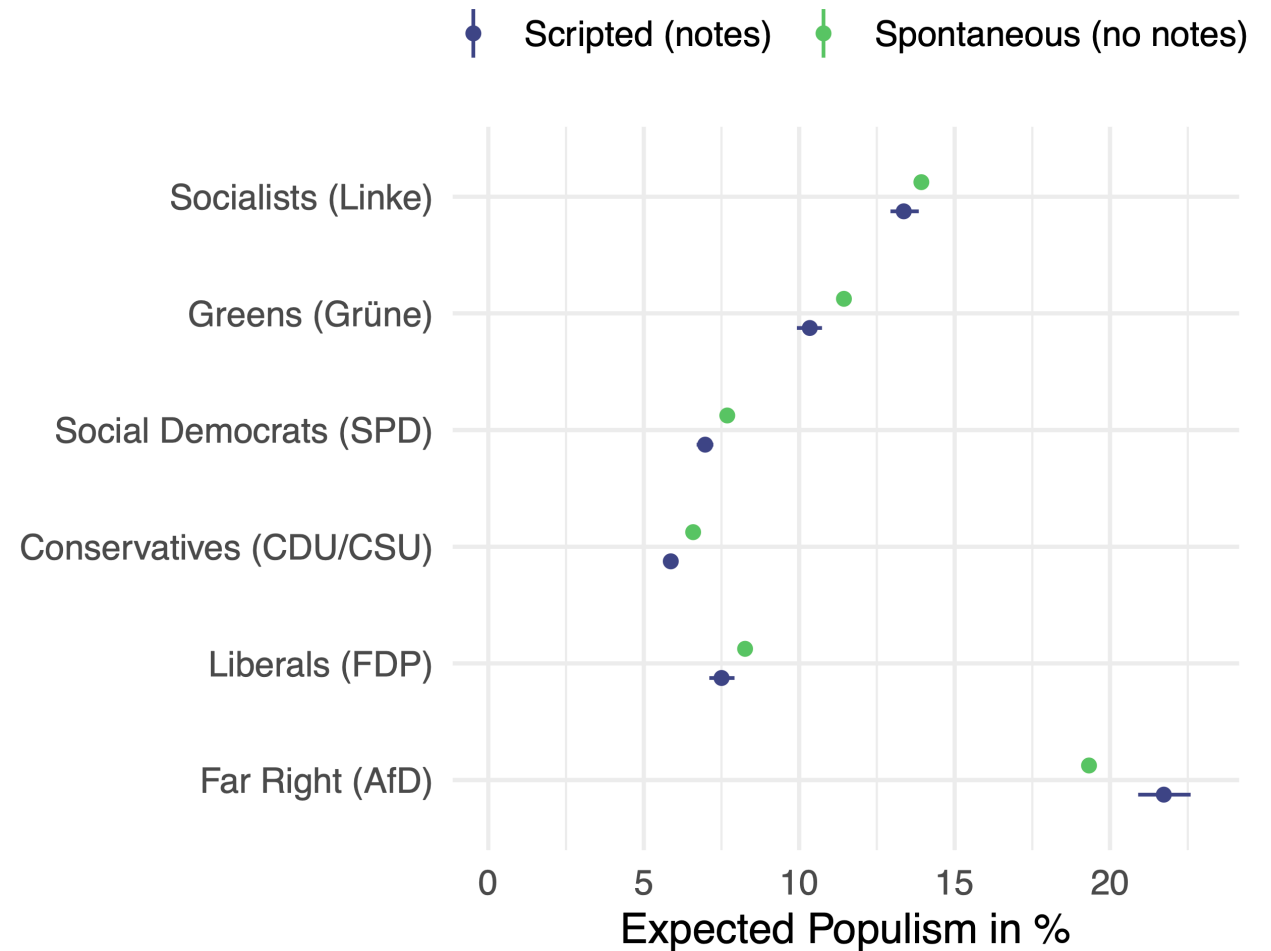
Research question

Does populist rhetoric emerge organically in unscripted speech, or is it prepared in advance?

Gaze direction reveals delivery mode: looking at notes = scripted; looking at audience = more *spontaneous*.

Key findings

- **Mainstream parties:** *spontaneous* → more populist language
- **Far-Right AfD reverses:** scripted → more populist language (populism is a deliberate strategy)



Conclusion

WMS are an implicit implementation of the replication framework the discipline has long discussed:

- **Automation** removes human error and undocumented execution order
- **Scalability** breaks the infrastructural constraint that keeps CSS artificially smaller
- **Portability** reduces environment inconsistency across machines and over time
- **Readability** makes pipelines comprehensible enough to be reviewed, verified, and built upon
- **Technical Documentation** creates a provenance record automatically
- **Analytical Documentation** makes the garden of forking paths visible and auditable

Together, they fulfil what *King (1995)* demanded: sufficient information for a third party to understand, evaluate, and extend the work.

Introducing Workflow Management to Computational Social Science

Andreas Kuepfer

TU Darmstadt

`andreas.kuepfer@tu-darmstadt.de`

`andreaskuepfer.github.io`

`ankuepfer.bsky.social`

Christian Arnold

University of Birmingham

`c.arnold.2@bham.ac.uk`

`chrisguarnold.github.io`

`chrisguarnold.bsky.social`

PARLIVIEW WORKSHOP | MAY 12, 2026 | UNIVERSITY COLLEGE DUBLIN, IRELAND